1. Introduction à JavaScript

Cours théorique - Séance 3

17 mars

---

2. JavaScript : le langage du web

JavaScript = Interactivité

C'est le seul langage de programmation que les navigateurs comprennent.

---

3. JavaScript ≠ Java

Deux langages complètement différents !

- **Java** : Langage backend, applications lourdes
- **JavaScript** : Langage web, s'exécute dans le navigateur

---

4. JavaScript fait 3 choses

- **Réagir aux actions** (clics, saisies...)
- **Modifier la page** (ajouter/supprimer du contenu)
- **Communiquer avec des serveurs** (récupérer des données)

---

5. Où écrire du JavaScript ?

- **Inline** (à éviter)
- **Interne** (pour tester)
- **Externe** (recommandé)

Même logique que CSS : séparer structure, présentation et comportement.

---

6. Lier un fichier JavaScript au HTML

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Ma page</title>
  <script src="script.js"></script>
</head>
<body>

</body>
</html>
```

---

7. La console : ton amie

```js
console.log('Bonjour !');
console.log(2 + 2);
```

```html
<h1>Bienvenue</h1><p>Ouvre la console (F12) et tape le snippet.</p>
```

---

8. DevTools (F12)

Trois onglets importants :

- **Console** : logs, erreurs, exécuter du JS
- **Elements** : inspecter le DOM en temps réel
- **Sources** : breakpoints, débogage

---

9. Qu'est-ce qu'une variable ?

Une **variable** = une boîte nommée qui stocke une valeur.

- On lui donne un **nom** (ex. `prenom` , `age`)
- Elle contient une **valeur** (texte, nombre, etc.)
- On peut la lire et parfois la modifier

---

10. const : variable constante

```
const PI = 3.14159;
const API_URL = 'https://api.exemple.com';

PI = 3.14;  // ❌ ERREUR !
```

---

11. const : quand l'utiliser ?

Règle : const par défaut

- Utilise `const` pour tout ce qui ne change pas
- Utilise `let` uniquement quand tu dois **réassigner** (compteur, boucle, etc.)
- Plus sûr : le code indique clairement ce qui peut changer

---

12. Constantes avec majuscules (UPPER_SNAKE_CASE)

```
// Constantes « vraies » (jamais modifiées) → MAJUSCULES
const PI = 3.14159;
const MAX_SCORE = 100;
const API_URL = 'https://api.exemple.com';

// Convention : UPPER_SNAKE_CASE (majuscules + underscores)
```

---

13. let : variable modifiable

```
let prenom = 'Alice';
let age = 25;
```

---

14. let : on peut réassigner

```
let score = 0;
console.log(score);  // 0

score = 10;
console.log(score);  // 10
```

---

15. var : à éviter

```
var nom = 'Bob';
// Ancien JS (avant 2015)
```

---

16. var vs let : la différence de scope

```
// var : scope « fonction » → la variable FUIT hors du bloc
if (true) {
  var x = 10;
}
console.log(x);  // 10 ! (x existe encore en dehors)

// let : scope « bloc » → la variable reste DANS le bloc
if (true) {
  let y = 10;
}
console.log(y);  // ReferenceError ! (y n'existe pas ici)

// Hoisting : var existe avant sa déclaration
console.log(z);  // undefined (pas d'erreur !)
var z = 5;
```

---

17. Règles de nommage (camelCase)

```
let prenomUtilisateur = 'Alice';
```

---

18. Nombres

```
let age = 25;
let prix = 19.99;
let temperature = -5;
```

---

19. Chaînes de caractères

```
let prenom = 'Alice';
let nom = "Dupont";
```

---

20. Template literals

```
let prenom = 'Alice';
let age = 25;

let message = \`Je m'appelle \${prenom} et j'ai \${age} ans.\`;
console.log(message);
```

---

21. Booléens

```
let isConnected = true;
let hasPermission = false;
```

---

22. Tableaux (index, length)

```
let fruits = ['Pomme', 'Banane', 'Orange'];

console.log(fruits[0]);  // 'Pomme'
console.log(fruits[1]);  // 'Banane'
console.log(fruits[2]);  // 'Orange'
console.log(fruits.length);  // 3
```

---

23. Objets (propriétés, accès)

```
let personne = {
    prenom: 'Alice',
    age: 25,
    ville: 'Paris'
};

console.log(personne.prenom);  // 'Alice'
console.log(personne['age']);  // 25
```

24. Opérateurs arithmétiques (+, -, *, /, %)

```
let a = 10;
let b = 3;

console.log(a + b);  // 13
console.log(a - b);  // 7
console.log(a * b);  // 30
console.log(a / b);  // 3.333...
console.log(a % b);  // 1 (reste)
```

25. Opérateurs de comparaison (== vs ===)

```
let x = 5;

x == 5     // true (valeur)
x == '5'   // true (conversion ⚠️)

x === 5    // true (valeur ET type)
x === '5'  // false (types différents)

// WTFJS:
"b" + "a" + + "a" + "a";  // ????

// Utilise TOUJOURS === et !== !
```

26. Opérateurs logiques (&&, ||, !)

```
let age = 20;
let hasLicense = true;
let jour = 'samedi';
let isConnected = true;

console.log(age >= 18 && hasLicense); // true

console.log(jour === 'samedi' || jour === 'dimanche'); // true

console.log(!isConnected); // false

console.log(!!isConnected); // ???
```

27. Condition if

```
let age = 20;

if (age >= 18) {
    console.log('Majeur');
}
```

28. if...else

```
let age = 16;

if (age >= 18) {
    console.log('Majeur');
} else {
    console.log('Mineur');
}
```

29. if...else if...else

```
let note = 15;

if (note >= 16) {
    console.log('Très bien');
} else if (note >= 12) {
    console.log('Bien');
} else if (note >= 10) {
    console.log('Passable');
} else {
    console.log('Insuffisant');
}
```

30. Opérateurs logiques dans les conditions

`&&` (ET), `||` (OU), `!` (NON)

```
if (age >= 18 && hasLicense) { ... }
if (jour === 'samedi' || jour === 'dimanche') { ... }
```

```
if (!isConnected) { ... }
```

---

31. try/catch — gestion d'erreurs basique

```
try {
    let data = JSON.parse('texte invalide');  // Lance une erreur
} catch (erreur) {
    console.log('Erreur attrapée :', erreur.message);
}
// Le programme continue au lieu de planter
```

---

32. Boucle for

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
// 0, 1, 2, 3, 4
```

---

33. Boucle while

```
let compteur = 0;

while (compteur < 3) {
    console.log(compteur);
    compteur++;
}
// 0, 1, 2
```

---

34. forEach — parcourir un tableau

```
let fruits = ['Pomme', 'Banane', 'Orange'];

fruits.forEach(function(fruit) {
    console.log(fruit);
});
// Pomme, Banane, Orange
```

35. map — transformer un tableau

```
let nombres = [1, 2, 3, 4, 5];

let doubles = nombres.map(function(n) {
    return n * 2;
});
console.log(doubles);  // [2, 4, 6, 8, 10]
```

```
<p>Ouvre la console et exécute le snippet.</p>
```

---

36. map vs forEach : résumé

map() transforme et retourne un nouveau tableau. **forEach()** exécute une action pour chaque élément (retourne `undefined`).

---

37. filter — filtrer un tableau

```
let nombres = [1, 2, 3, 4, 5];

let pairs = nombres.filter(function(n) {
    return n % 2 === 0;
});
console.log(pairs);  // [2, 4]
```

---

38. find — trouver un élément

```
let fruits = ['Pomme', 'Banane', 'Orange'];

let result = fruits.find(function(fruit) {
    return fruit === 'Banane';
});
console.log(result);  // 'Banane'
```

---

39. null vs undefined

```
// undefined = « pas de valeur » (JS l'assigne automatiquement)
let x;
console.log(x);  // undefined

let fruits = ['Pomme', 'Banane'];
let result = fruits.find(f => f === 'Cerise');
console.log(result);  // undefined (aucun élément trouvé)

// null = « valeur intentionnellement vide »
let user = null;  // Pas d'utilisateur connecté

// Vérifier avant d'utiliser
if (result !== undefined && result !== null) {
    console.log(result);
}
```

---

40. Fonctions : réutiliser du code

```
function direBonjour() {
    console.log('Bonjour !');
}

direBonjour();  // 'Bonjour !'
```

---

41. Fonctions avec paramètres

```
function direBonjour(prenom) {
    console.log(\`Bonjour \${prenom} !\`);
}

direBonjour('Alice');   // 'Bonjour Alice !'
direBonjour('Bob');     // 'Bonjour Bob !'
```

---

42. Fonctions qui retournent une valeur

```
function addition(a, b) {
    return a + b;
}
```

```
let resultat = addition(5, 3);
console.log(resultat);  // 8
```

---

43. Fonctions fléchées (arrow functions)

```
// Fonction classique
function addition(a, b) {
    return a + b;
}

// Fonction fléchée
const addition = (a, b) => {
    return a + b;
};

// Version ultra-courte
const addition = (a, b) => a + b;
```

---

44. Le DOM : Document Object Model

Le DOM = représentation de ta page HTML en JavaScript.

Le navigateur transforme ton HTML en un arbre d'objets.

---

45. Visualiser le DOM (arbre)

DevTools (F12) → Elements : tu vois l'arbre HTML.

---

46. getElementById

```
<h1 id="titre">Mon titre</h1>

let titre = document.getElementById('titre');
console.log(titre);
```

---

47. querySelector et querySelectorAll

```
// querySelector : premier élément
let paragraphe = document.querySelector('.texte');

// querySelectorAll : tous les éléments
let items = document.querySelectorAll('.item');
items.forEach(function(item) { ... });
```

---

48. Modifier le contenu : textContent

```
let titre = document.getElementById('titre');
titre.textContent = 'Nouveau titre';
```

---

49. Modifier le HTML : innerHTML (attention XSS)

```
let conteneur = document.getElementById('conteneur');
conteneur.innerHTML = '<p>Nouveau <strong>paragraphe</strong></p>';

// ⚠️ Attention XSS si contenu utilisateur !
```

---

50. Modifier les styles : style

```
let texte = document.getElementById('texte');
texte.style.color = 'red';
texte.style.fontSize = '24px';
```

---

51. classList (add, remove, toggle)

```
let box = document.getElementById('box');

box.classList.add('active');
box.classList.remove('container');
box.classList.toggle('visible');
```

---

52. createElement

```
let nouveauParagraphe = document.createElement('p');
nouveauParagraphe.textContent = 'Nouveau contenu';
```

---

53. appendChild et remove

```
let conteneur = document.getElementById('liste');

conteneur.appendChild(nouveauParagraphe);

// Supprimer : récupérer l'élément à supprimer
let element = document.getElementById('elementASupprimer');
element.remove();
```

---

54. Événements : réagir aux actions

Événement = action de l'utilisateur (clic, saisie...)

`addEventListener` = « écoute cet événement »

---

55. addEventListener

```
let bouton = document.getElementById('monBouton');

bouton.addEventListener('click', function() {
    console.log('Bouton cliqué !');
});
```

---

56. Événement click

```
let btn = document.getElementById('btn');
let message = document.getElementById('message');

btn.addEventListener('click', function() {
    message.textContent = 'Tu as cliqué !';
});
```

57. Événement input

```
let prenom = document.getElementById('prenom');
let affichage = document.getElementById('affichage');

prenom.addEventListener('input', function() {
    affichage.textContent = \`Bonjour \${prenom.value} !\`;
});
```

58. Événement submit

```
let form = document.getElementById('monForm');

form.addEventListener('submit', function(event) {
    event.preventDefault();
    console.log('Formulaire envoyé !');
});
```

59. event.preventDefault()

```
form.addEventListener('submit', function(event) {
    event.preventDefault();  // Empêche le rechargement
    // Traiter les données ici
});
```

60. Objet event (target, type, etc.)

```
bouton.addEventListener('click', function(event) {
    console.log(event.type);    // 'click'
    console.log(event.target); // L'élément cliqué
    console.log(event.clientX); // Position X souris
});
```

61. Récupérer les valeurs d'un formulaire

```
let form = document.getElementById('monForm');
let inputNom = document.getElementById('nom');

form.addEventListener('submit', function(e) {
    e.preventDefault();
    let valeur = inputNom.value;
    console.log(valeur);
});
```

---

62. Validation basique (champ vide, longueur)

```
let nom = inputNom.value.trim();

if (nom === '') {
    // Champ vide
}

if (nom.length < 3) {
    // Trop court
}
```

---

63. Afficher les messages d'erreur

```
let nom = inputNom.value.trim();

if (nom === '') {
    // Champ vide
}

if (nom.length < 3) {
    // Trop court
}
```

---

64. Exemple : formulaire avec validation

```
let inputNom = document.getElementById('nom');
let erreurNom = document.getElementById('erreur-nom');
let btn = document.getElementById('btnEnvoyer');
```

```
btn.addEventListener('click', function() {
  erreurNom.textContent = '';
  let nom = inputNom.value.trim();
  if (nom === '') {
    erreurNom.textContent = 'Le nom est requis';
    return;
  }
  if (nom.length < 3) {
    erreurNom.textContent = 'Le nom doit faire au moins 3 caractères';
    return;
  }
  alert('Bienvenue, ' + nom + ' !');
});
```

```
<form id="formulaire">
  <input type="text" id="nom" placeholder="Nom">
  <span id="erreur-nom"></span>
  <button type="button" id="btnEnvoyer">Envoyer</button>
</form>
```

```
#erreur-nom { color: red; font-size: 0.9rem; display: block; margin-top:
4px; }
```

```
let inputNom = document.getElementById('nom');
let erreurNom = document.getElementById('erreur-nom');
let btn = document.getElementById('btnEnvoyer');

btn.addEventListener('click', function() {
  erreurNom.textContent = '';
  let nom = inputNom.value.trim();
  if (nom === '') {
    erreurNom.textContent = 'Le nom est requis';
    return;
  }
  if (nom.length < 3) {
    erreurNom.textContent = 'Le nom doit faire au moins 3 caractères';
    return;
  }
  alert('Bienvenue, ' + nom + ' !');
});
```

À taper dans la console :

```
// Teste le formulaire dans la preview :
// 1. Vide → "Le nom est requis"
```

```
// 2. "ab" → "au moins 3 caractères"
// 3. "Alice" → "Bienvenue, Alice !"
```

---

65. Compteur de clics

```
// index.html


// script.js
```

```
<button id="btnCompter">Clique-moi</button>
<p>Clics : <span id="compteur">0</span></p>
```

```
let btn = document.getElementById('btnCompter');
let compteur = document.getElementById('compteur');
let count = 0;
btn.addEventListener('click', function() {
  count++;
  compteur.textContent = count;
});
```

---

66. Toggle de visibilité

```
// index.html


// style.css


// script.js
```

```
<button id="btnToggle">Afficher/Masquer</button>
<p id="texte">Je suis visible !</p>
```

```
.hidden { display: none; }
```

```
let btn = document.getElementById('btnToggle');
let texte = document.getElementById('texte');
btn.addEventListener('click', function() {
```

```
  texte.classList.toggle('hidden');
});
```

---

67. To-do list minimaliste

```
// index.html


// script.js
```

```
<h2>Ma To-Do List</h2>
<input type="text" id="nouvelle-tache" placeholder="Nouvelle tâche...">
<button id="btn-ajouter">Ajouter</button>
<ul id="liste-taches"></ul>
```

```
h2 { margin-bottom: 12px; }
input { padding: 10px; font-size: 16px; width: 220px; }
ul { list-style: none; padding: 0; margin-top: 18px; }
li { padding: 10px; margin: 6px 0; border-radius: 6px; cursor: pointer;
background: #f0f0f0; }
li:hover { background: #e0e0e0; }
```

```
let inputTache = document.querySelector('#nouvelle-tache');
let btnAjouter = document.querySelector('#btn-ajouter');
let listeTaches = document.querySelector('#liste-taches');
btnAjouter.addEventListener('click', function() {
  let texte = inputTache.value;
  if (texte !== '') {
    let li = document.createElement('li');
    li.textContent = texte;
    li.addEventListener('click', function() { li.remove(); });
    listeTaches.appendChild(li);
    inputTache.value = '';
  }
});
```

---

68. Bonnes pratiques JavaScript

- Utilise `const` par défaut, `let` si la valeur change
- Nomme tes variables clairement (camelCase)

- Cache tes sélecteurs DOM dans des variables
- Utilise `textContent` plutôt que `innerHTML` si possible
- Valide toujours les entrées utilisateur

---

69. Erreurs courantes

- Oublier de sélectionner l'élément avant de le modifier
- Script dans le `<head>` avant que le DOM existe → placer avant `</body>`
- Utiliser `==` au lieu de `===`

---

70. Débogage : console.log et DevTools

`console.log` partout pour comprendre ce qui se passe !

- **Console** : logs, erreurs
- **Elements** : inspecter le DOM
- **Sources** : breakpoints

---

71. Récap JavaScript

- Variables (`let` , `const`)
- Types (Number, String, Boolean, Array, Object)
- Conditions, boucles, fonctions
- DOM (sélection, modification)
- Événements (`click` , `input` , `submit`)

---

72. Prochaine fois : TP

TP « Dynamiser l'interface utilisateur »

- Menu burger interactif
- To-do list dynamique
- Système de filtres

Même boîte à outils : querySelector, événements, conditions, boucles.

---

73. Ressources

- MDN Web Docs
- JavaScript.info
- JavaScript30